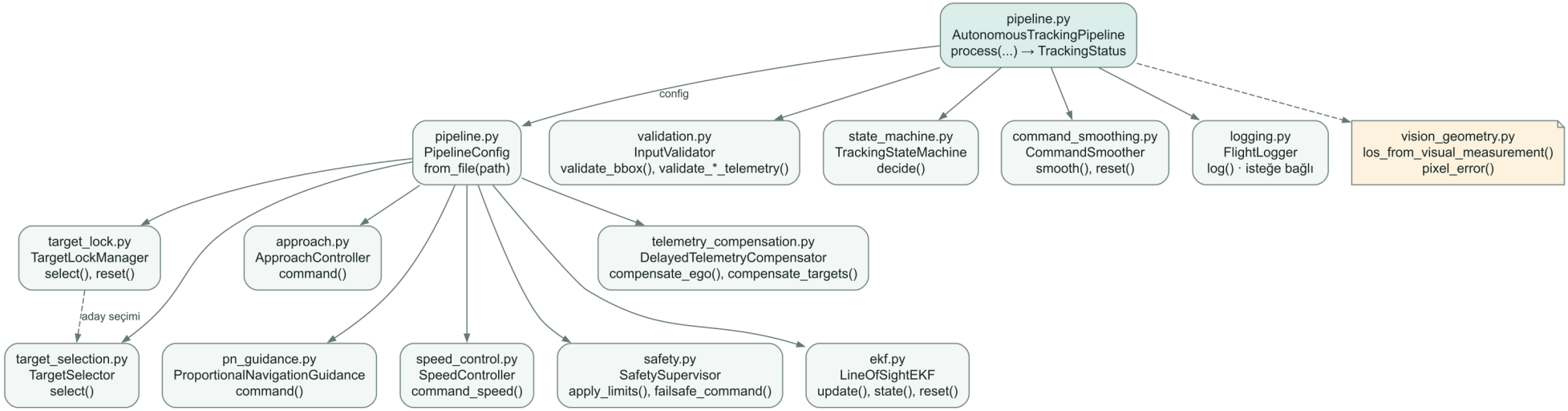
Otonom Takip Sistemi Mimari Diyagramları

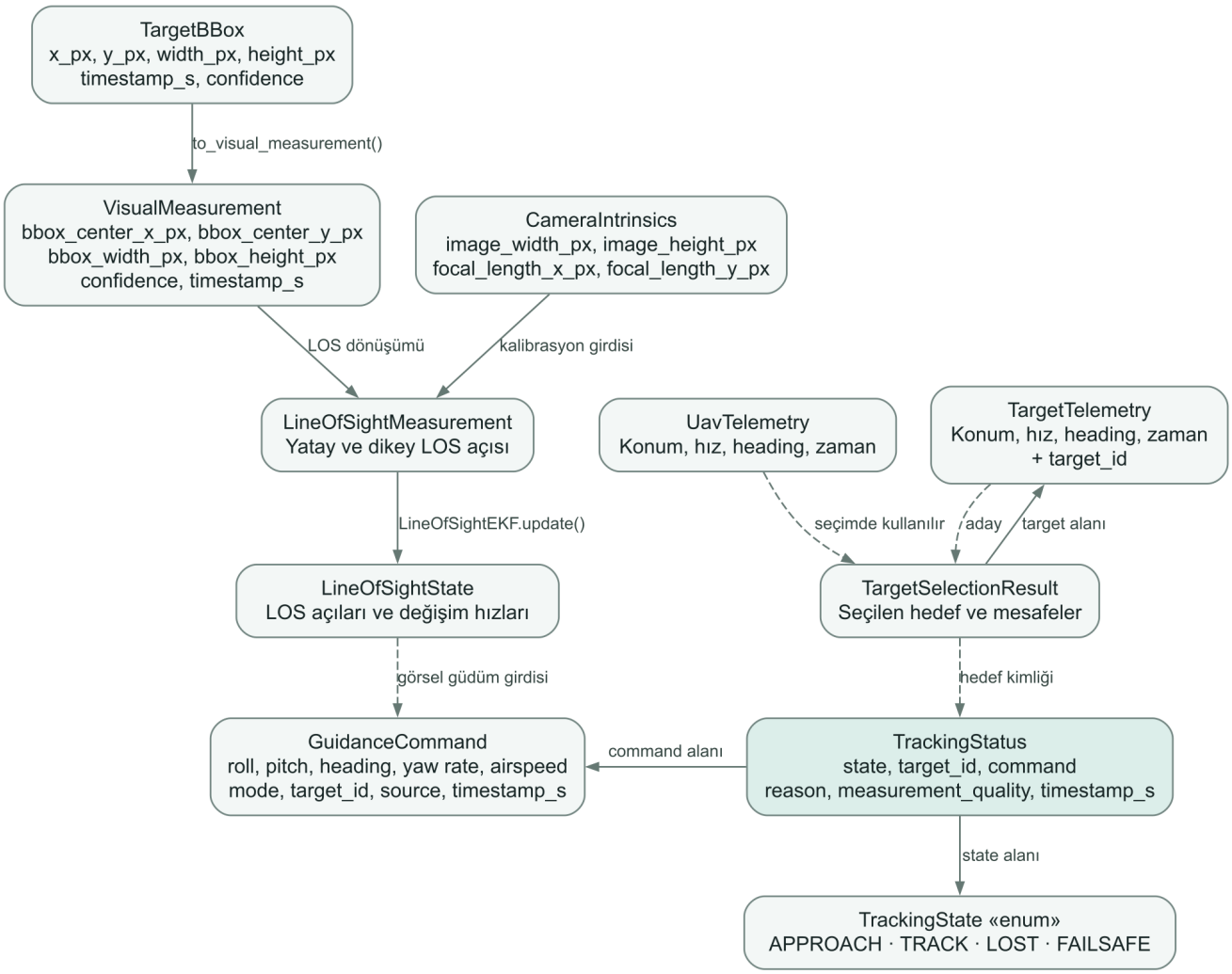
Sınıf ilişkileri, veri modelleri ve çalışma akışı. Başlangıç noktası AutonomousTrackingPipeline.process(); her çağrı TrackingStatus döndürür.

1 · Çekirdek sınıflar ve bağımlılıklar



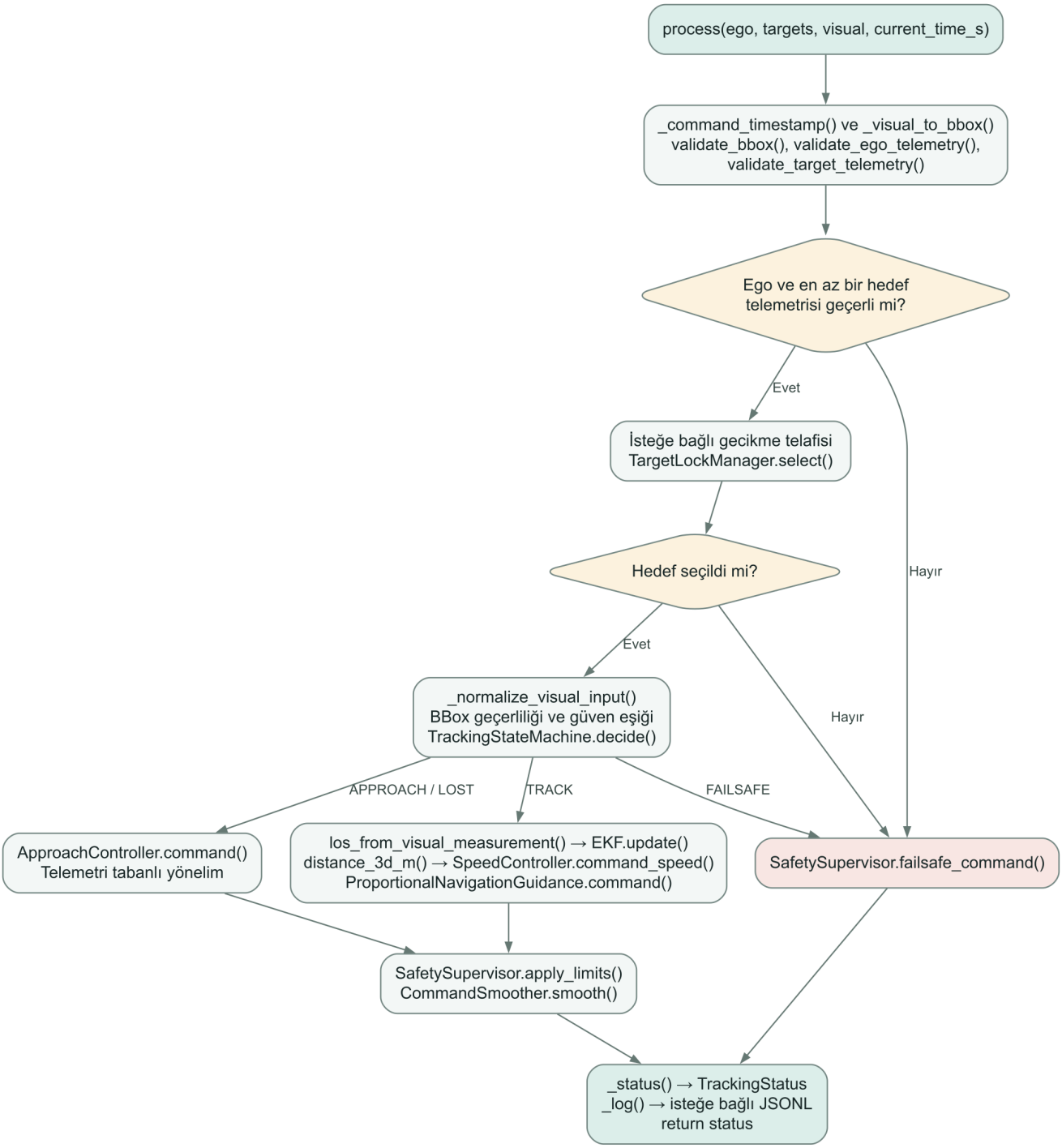
Düz ok tutulan bileşeni, kesikli ok çağrı veya kullanım bağımlılığını gösterir. Modül adı sınıfın üstünde, başlıca metotlar altında verilmiştir.

2 · Veri sınıfları ve dönüşümler



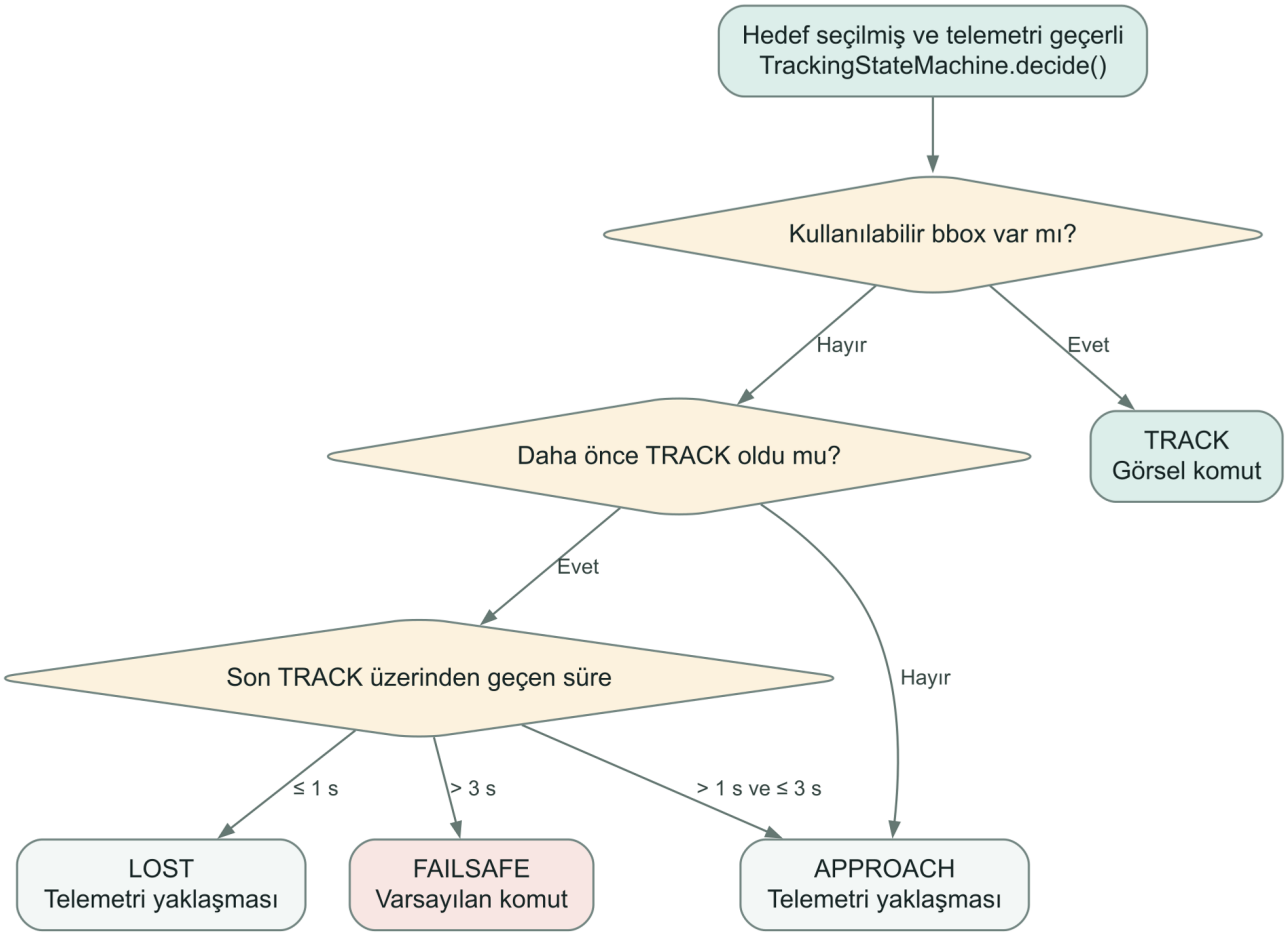
Kutular seçili alanları gösterir. Kutu verisinde hedef kimliği yoktur; seçilen telemetri hedefiyle eşleşmesi dış girdinin varsayımıdır.

3 · process() fonksiyon akışı



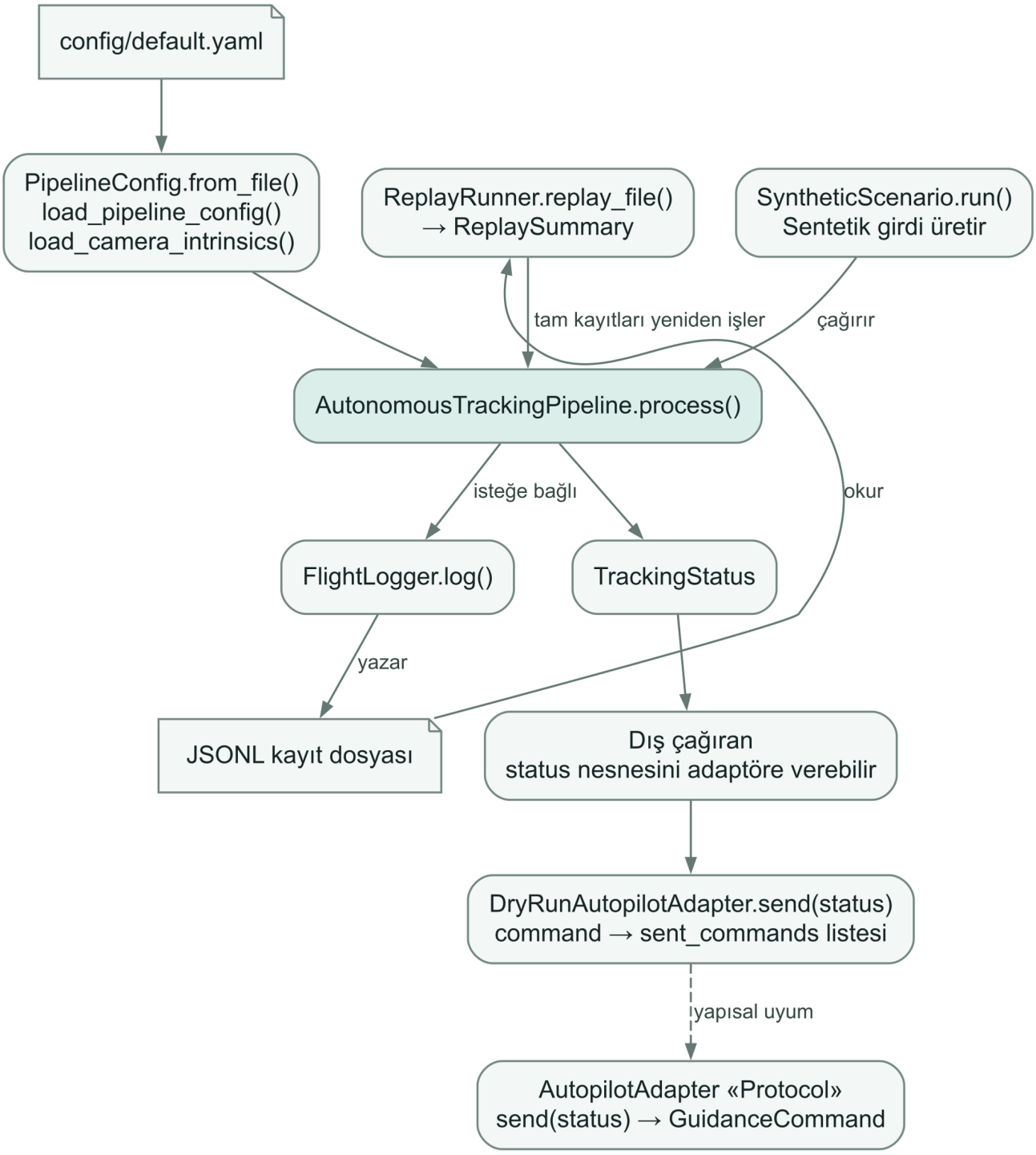
Telemetri telafisi etkinse seçimden önce çalışır. FAILSAFE yolu normal sınırlandırma ve yumuşatmayı atlar. Logger yalnızca log_path ayarlandığında kayıt yazar.

4 · Durum kararı



Eşikler varsayılan değerlerdir ve yapılandırmadan değişebilir. Telemetri/aday hatasındaki erken FAILSAFE, pipeline çıktısıdır; o çağrıda decide() çalışmaz. Son TRACK zamanı, geçerli bbox ile yenilenir.

5 · Yapılandırma, kayıt ve dış kullanım



DryRun, send() imzasıyla AutopilotAdapter protokolüne yapısal olarak uyar; sınıftan kalıtım almaz ve MAVLink göndermez. Pipeline adaptörü kendisi çağırmaz.

6 · Sınıf, fonksiyon ve metot envanteri

autonomous_tracking/ paketinden AST ile çıkarıldı: 32 sınıf ve 10 modül fonksiyonu. Sınıf metotları, özellikler ve özel yardımcılar ilgili modülün altında gösterilir.

approach.py

Yerel modül bağımlılıkları: geo, models

`class ApproachController satır 9`

`ApproachController.__init__(max_roll_deg: float=30.0, max_pitch_deg: float=12.0, heading_to_roll_gain: float=0.45, altitude_to_pitch_gain: float=0.015, approach_airspeed_mps: float=24.0) -> None satır 10`

`ApproachController.command(ego: UavTelemetry, target: TargetTelemetry, timestamp_s: float) -> GuidanceCommand satır 24`

autopilot_adapter.py

Yerel modül bağımlılıkları: models

`class AutopilotAdapter satır 14`

`AutopilotAdapter.send(status: TrackingStatus) -> GuidanceCommand satır 15`

`class DryRunAutopilotAdapter satır 19`

`DryRunAutopilotAdapter.__init__() -> None satır 20`

`DryRunAutopilotAdapter.send(status: TrackingStatus) -> GuidanceCommand satır 23`

camera_calibration.py

Yerel modül bağımlılıkları: models

`load_camera_intrinsics(data: dict[str, Any]) -> CameraIntrinsics satır 10`

command_smoothing.py

Yerel modül bağımlılıkları: geo, models

`class CommandSmoother satır 11`

`CommandSmoother.__init__(max_delta_roll_deg: float=8.0, max_delta_pitch_deg: float=5.0, max_delta_airspeed_mps: float=3.0) -> None satır 12`

`CommandSmoother.reset() -> None satır 23`

`CommandSmoother.smooth(command: GuidanceCommand) -> GuidanceCommand satır 26`

`CommandSmoother._limit_delta(value: float, previous: float, max_delta: float) -> float satır 51`

config.py

Yerel modül bağımlılıkları: yok

`load_pipeline_config(path: str | Path | None=None) -> dict[str, Any] satır 14`

ekf.py

Yerel modül bağımlılıkları: vision_geometry

`class LineOfSightState satır 14`

`class LineOfSightEKF satır 22`

`LineOfSightEKF.__init__(process_noise: float=0.08, measurement_noise: float=0.035, initial_variance: float=1.0) -> None satır 25`

`LineOfSightEKF.reset() -> None satır 38`

`LineOfSightEKF.update(measurement: LineOfSightMeasurement, telemetry_los: Optional[LineOfSightMeasurement]=None) -> LineOfSightState satır 43`

`LineOfSightEKF.state -> LineOfSightState satır 68`

`LineOfSightEKF._predict(dt: float) -> None satır 79`

`LineOfSightEKF._correct(measurement: LineOfSightMeasurement, noise: float) -> None satır 94`

geo.py

Yerel modül bağımlılıkları: models

`clamp(value: float, lower: float, upper: float) -> float satır 12`

wrap_degrees(angle_deg: float) -> float satır 16

horizontal_distance_m(a: UavTelemetry, b: TargetTelemetry) -> float satır 22

distance_3d_m(a: UavTelemetry, b: TargetTelemetry) -> float satır 31

bearing_deg(a: UavTelemetry, b: TargetTelemetry) -> float satır 37

destination_point(latitude_deg: float, longitude_deg: float, distance_m: float, heading_deg: float) -> tuple[float, float] satır 46

logging.py

Yerel modül bağımlılıkları: yok

class FlightLogger satır 12

FlightLogger.__init__(path: str | Path) -> None satır 13

FlightLogger.log(record: dict[str, Any]) -> None satır 17

FlightLogger._jsonable(value: Any) -> Any satır 21

models.py

Yerel modül bağımlılıkları: yok

class UavTelemetry satır 21

class TargetTelemetry satır 31

class VisualMeasurement satır 42

VisualMeasurement.bbox_area_px -> float satır 51

class TargetBBox satır 56

TargetBBox.to_visual_measurement() -> VisualMeasurement satır 70

class CameraIntrinsics satır 82

CameraIntrinsics.from_fov(image_width_px: int, image_height_px: int, horizontal_fov_deg: float, vertical_fov_deg: Optional[float]=None) -> 'CameraIntrinsics' satır 89

class GuidanceCommand satır 116

class TrackingState satır 128

class TrackingStatus satır 136

pipeline.py

Yerel modül bağımlılıkları: approach, camera_calibration, command_smoothing, config, ekf, geo, logging, models, pn_guidance, safety, speed_control, state_machine, target_lock, target_selection, telemetry_compensation, validation, vision_geometry

class PipelineConfig satır 38

PipelineConfig.from_file(path: Union[str, Path]) -> 'PipelineConfig' satır 63

class AutonomousTrackingPipeline satır 127

AutonomousTrackingPipeline.__init__(config: PipelineConfig) -> None satır 128

AutonomousTrackingPipeline.process(ego: UavTelemetry, targets: Iterable[TargetTelemetry], visual: Optional[Union[TargetBBox, VisualMeasurement]]=None, current_time_s: Optional[float]=None) -> TrackingStatus satır 148

AutonomousTrackingPipeline._normalize_visual_input(visual: Optional[Union[TargetBBox, VisualMeasurement]]) -> Optional[VisualMeasurement] satır 298

AutonomousTrackingPipeline._visual_to_bbox(visual: Optional[Union[TargetBBox, VisualMeasurement]]) -> Optional[TargetBBox] satır 308

AutonomousTrackingPipeline._command_timestamp(ego: UavTelemetry, visual: Optional[Union[TargetBBox, VisualMeasurement]], current_time_s: Optional[float]) -> float satır 325

AutonomousTrackingPipeline._status(state: TrackingState, target_id: Optional[str], command: GuidanceCommand, reason: Optional[str], measurement_quality: float, timestamp_s: float, validation: ValidationResult) -> TrackingStatus satır 337

AutonomousTrackingPipeline._log(status: TrackingStatus, bbox_validation: ValidationResult, ego: UavTelemetry, targets: list[TargetTelemetry], bbox: Optional[TargetBBox]) -> None satır 356

pn_guidance.py

Yerel modül bağımlılıkları: ekf, geo, models

class ProportionalNavigationGuidance satır 12

ProportionalNavigationGuidance.__init__(navigation_constant: float=3.0, los_angle_gain: float=70.0, los_rate_gain: float=28.0, max_roll_deg: float=35.0, max_pitch_deg: float=15.0, max_yaw_rate_deg_s: float=25.0) -> None satır 13

ProportionalNavigationGuidance.command(ego: UavTelemetry, target: TargetTelemetry, los: LineOfSightState, airspeed_mps: float) -> GuidanceCommand satır 29

replay.py

Yerel modül bağımlılıkları: models, pipeline

class ReplaySummary satır 16

class ReplayRunner satır 23

ReplayRunner.__init__(pipeline: AutonomousTrackingPipeline) -> None satır 24

ReplayRunner.from_config(config: PipelineConfig) -> 'ReplayRunner' satır 28

ReplayRunner.replay_file(path: str | Path) -> ReplaySummary satır 31

ReplayRunner._process_record(record: dict[str, Any]) -> str satır 55

safety.py

Yerel modül bağımlılıkları: geo, models

class SafetySupervisor satır 11

SafetySupervisor.__init__(max_roll_deg: float=35.0, max_pitch_deg: float=15.0, max_yaw_rate_deg_s: float=25.0, min_airspeed_mps: float=16.0, max_airspeed_mps: float=30.0, safe_airspeed_mps: float=22.0, close_distance_m: float=45.0) -> None satır 12

SafetySupervisor.apply_limits(command: GuidanceCommand, target_distance_m: float | None=None) -> GuidanceCommand satır 30

SafetySupervisor.failsafe_command(ego: UavTelemetry, timestamp_s: float, reason: str) -> GuidanceCommand satır 50

simulation.py

Yerel modül bağımlılıkları: models, pipeline

class SyntheticScenario satır 11

SyntheticScenario.__init__(pipeline: AutonomousTrackingPipeline, duration_s: float=30.0, dt_s: float=0.2) -> None satır 12

SyntheticScenario.run() -> list[TrackingStatus] satır 22

SyntheticScenario._bbox_for_time(timestamp_s: float) -> TargetBBox | None satır 43

speed_control.py

Yerel modül bağımlılıkları: geo, models

class SpeedController satır 11

SpeedController.__init__(min_airspeed_mps: float=16.0, cruise_airspeed_mps: float=22.0, max_airspeed_mps: float=30.0, far_distance_m: float=500.0, near_distance_m: float=90.0, small_bbox_ratio: float=0.004, large_bbox_ratio: float=0.08) -> None satır 12

SpeedController.command_speed(distance_m: Optional[float], visual: Optional[VisualMeasurement], camera: Optional[CameraIntrinsics]) -> float satır 30

state_machine.py

Yerel modül bağımlılıkları: models

class TrackingStateMachine satır 8

TrackingStateMachine.__init__(lost_timeout_s: float=1.0, reacquire_timeout_s: float=3.0) -> None satır 9

TrackingStateMachine.decide(has_target: bool, telemetry_valid: bool, bbox_valid: bool, current_time_s: float) -> TrackingState satır 15

target_lock.py

Yerel modül bağımlılıkları: models, target_selection

class TargetLockManager satır 11

TargetLockManager.__init__(switch_distance_ratio: float=0.75, switch_confirmations: int=2) -> None satır 12

TargetLockManager.reset() -> None satır 23

TargetLockManager.select(ego: UavTelemetry, targets: Iterable[TargetTelemetry], selector: TargetSelector) -> TargetSelectionResult | None satır 28

TargetLockManager._locked_result(ego: UavTelemetry, targets: list[TargetTelemetry], selector: TargetSelector) -> TargetSelectionResult | None satır 64

target_selection.py

Yerel modül bağımlılıkları: geo, models

class TargetSelectionResult satır 13

class TargetSelector satır 20

TargetSelector.__init__(max_distance_m: Optional[float]=None, max_altitude_difference_m: Optional[float]=None) -> None satır 21

TargetSelector.select(ego: UavTelemetry, targets: Iterable[TargetTelemetry]) -> Optional[TargetSelectionResult] satır 29

telemetry_compensation.py

Yerel modül bağımlılıkları: geo, models

class DelayedTelemetryCompensator satır 12

DelayedTelemetryCompensator.__init__(max_prediction_dt_s: float=2.0) -> None satır 20

DelayedTelemetryCompensator.compensate_ego(telemetry: UavTelemetry, current_time_s: float) -> UavTelemetry satır 23

DelayedTelemetryCompensator.compensate_target(telemetry: TargetTelemetry, current_time_s: float) -> TargetTelemetry satır 38

DelayedTelemetryCompensator.compensate_targets(targets: Iterable[TargetTelemetry], current_time_s: float) -> list[TargetTelemetry] satır 57

DelayedTelemetryCompensator._prediction_dt(telemetry_time_s: float, current_time_s: float) -> float satır 64

validation.py

Yerel modül bağımlılıkları: models

class ValidationResult satır 11

class InputValidator satır 17

InputValidator.__init__(max_telemetry_age_s: float=2.0, max_bbox_age_s: float=0.5, min_bbox_area_ratio: float=0.0001, max_bbox_area_ratio: float=0.75) -> None satır 18

InputValidator.validate_ego_telemetry(telemetry: UavTelemetry, current_time_s: float) -> ValidationResult satır 30

InputValidator.validate_target_telemetry(telemetry: TargetTelemetry, current_time_s: float) -> ValidationResult satır 37

InputValidator.validate_bbox(bbox: TargetBBox | None, camera: CameraIntrinsics, current_time_s: float) -> ValidationResult satır 44

InputValidator._validate_telemetry_age(telemetry_time_s: float, current_time_s: float) -> ValidationResult satır 72

vision_geometry.py

Yerel modül bağımlılıkları: models

class PixelError satır 12

class LineOfSightMeasurement satır 18

pixel_error(measurement: VisualMeasurement, camera: CameraIntrinsics) -> PixelError satır 24

los_from_visual_measurement(measurement: VisualMeasurement, camera: CameraIntrinsics) -> LineOfSightMeasurement satır 36